

Proyecto de Simulación

Sistema de Tiempo Real: Controlador Simplificado de Bomba de Infusión

Especificación DEVS de los Modelos Atómicos

Jhonatan Calle Galeano

Hernan Jara

1. Generador de Órdenes Médicas

El modelo **GeneradorOrdenes** es un modelo atómico autónomo (sin entradas) que genera periódicamente eventos `ordenMedica`, indicando el caudal objetivo que debe administrar la bomba. Las órdenes pueden generarse de manera determinística o estocástica según los escenarios definidos.

```
atomic GeneradorOrdenes is <Y, S,  $\delta_{int}$ ,  $\lambda$ , ta> where
S is
    caudalProximo :  $\mathbb{R}$ ;      -- caudal objetivo (0 a 200 ml/h)
    interarribo   : Time;    -- tiempo hasta la siguiente orden
     $\sigma$           : Time;    -- variable de avance de tiempo
end S
Y is
    ordenMedica :  $\mathbb{R}$ ;      -- puerto de salida: valor del caudal
end Y
 $\delta_{int}$ (caudalProximo, interarribo,  $\sigma$ ) is
    -- A definir en la implementacion: puede ser deterministica
    -- (valores fijos) o estocastica (usando distribuciones)
    caudalProximo = next_caudal_logic();
    interarribo   = next_time_logic();
     $\sigma$  = interarribo;
end  $\delta_{int}$ 
 $\lambda$ (caudalProximo, interarribo,  $\sigma$ ) is
    -- Emite el caudal objetivo por el puerto de salida
    ordenMedica = caudalProximo;
end  $\lambda$ 
ta(caudalProximo, interarribo,  $\sigma$ ) is
     $\sigma$ ;
end ta
end atomic
```

Figura 1: Especificación CML-DEVS: Generador de Órdenes Médicas

2. Sensor de Flujo

El modelo **SensorFlujo** reporta periódicamente el caudal real cada 1 segundo. Cuando recibe una actualización del actuador, incorpora ese valor para que la próxima medición refleje el estado físico real de la bomba.

```
atomic SensorFlujo is <X, Y, S,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta> where
S is
    caudalMedido  :  $\mathbb{R}$ ;  -- valor de caudal a reportar
    caudalActuador :  $\mathbb{R}$ ;  -- ultimo valor recibido del actuador
     $\sigma$           : Time; -- controla el periodo de muestreo (1 s)
end S
X is
    caudalActual :  $\mathbb{R}$ ;  -- caudal fisico recibido del actuador
end X
Y is
    sensorFlujo  :  $\mathbb{R}$ ;  -- medicion enviada al Controlador
end Y
 $\delta_{ext}((\text{caudalMedido}, \text{caudalActuador}, \sigma), e, (\text{puerto}, \text{valor}))$  is
    caudalActuador = valor;
     $\sigma = \sigma - e$ ;
    if (puerto = caudalActual)
end  $\delta_{ext}$ 
 $\delta_{int}(\text{caudalMedido}, \text{caudalActuador}, \sigma)$  is
    -- Si hay valor del actuador lo usa, sino genera internamente
    caudalMedido = next_caudal_logic(caudalActuador);
     $\sigma = 1.0$ ;
end  $\delta_{int}$ 
 $\lambda(\text{caudalMedido}, \text{caudalActuador}, \sigma)$  is
    sensorFlujo = caudalMedido;
end  $\lambda$ 
ta(caudalMedido, caudalActuador,  $\sigma$ ) is
     $\sigma$ ;
end ta
end atomic
```

Figura 2: Especificación CML-DEVS: Sensor de Flujo

Análisis del modelo

1. **Frecuencia de muestreo** (ta y δ_{int}): El sensor reporta el caudal cada 1 segundo. σ se resetea en 1.0 en cada transición interna.
2. **Incorporación del valor del actuador** (δ_{ext}): Al recibir `caudalActual` desde el actuador, el modelo almacena el valor en `caudalActuador` y mantiene el cronograma con $\sigma = \sigma - e$. En la próxima transición interna, `next_caudal_logic(caudalActuador)` utiliza ese valor para generar la medición.
3. **Estado inicial** (S): Se asume `caudalMedido = 0,0`, `caudalActuador = 0,0` y $\sigma = 1,0$.

3. Actuador de la Bomba

El modelo **ActuadorBomba** representa la interfaz física de la bomba: recibe comandos lógicos del controlador y, tras una latencia de 0.5 segundos, actualiza el caudal real suministrado al paciente.

```
atomic ActuadorBomba is <X, Y, S,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta> where
S is
  caudalFisico    :  $\mathbb{R}$ ;          -- caudal que actualmente sale de la bomba
  caudalObjetivo  :  $\mathbb{R}$ ;          -- caudal a alcanzar tras la latencia
   $\sigma$            : Time;         -- controla la latencia de 0.5 segundos
end S
X is
  ajustarCaudal   :  $\mathbb{R}$ ;          -- nuevo valor de dosis desde el controlador
  detenerBomba    : {signal};      -- senal de parada de emergencia
end X
Y is
  caudalActual    :  $\mathbb{R}$ ;          -- notifica el cambio fisico al Sensor
end Y
 $\delta_{ext}((\text{caudalFisico}, \text{caudalObjetivo}, \sigma), e, (\text{puerto}, \text{valor}))$  is
  defcases
    caudalObjetivo = valor;    if (puerto = ajustarCaudal)
     $\sigma = 0.5$ ;
    caudalObjetivo = 0.0;     if (puerto = detenerBomba)
     $\sigma = 0.5$ ;
  end defcases
end  $\delta_{ext}$ 
 $\delta_{int}(\text{caudalFisico}, \text{caudalObjetivo}, \sigma)$  is
  -- Cumplida la latencia, el caudal fisico alcanza al objetivo
  caudalFisico = caudalObjetivo;
   $\sigma$         = infinity;
end  $\delta_{int}$ 
 $\lambda(\text{caudalFisico}, \text{caudalObjetivo}, \sigma)$  is
  -- Emite el nuevo caudal real antes de pasar al estado pasivo
  caudalActual = caudalObjetivo;
end  $\lambda$ 
ta(caudalFisico, caudalObjetivo,  $\sigma$ ) is
   $\sigma$ ;
end ta
end atomic
```

Figura 3: Especificación CML-DEVS: Actuador de la Bomba

Análisis del modelo

1. **Manejo de la latencia** (δ_{ext} y ta): Al recibir un evento externo, el modelo guarda el nuevo valor en **caudalObjetivo** y establece $\sigma = 0,5$, obligando al sistema a esperar exactamente el tiempo de latencia especificado antes de aplicar el cambio.
2. **Estado pasivo**: Cuando no hay órdenes pendientes, $\sigma = \infty$. El actuador

mantiene el caudal actual indefinidamente hasta que el controlador envíe un nuevo evento.

3. **Sincronización con el sensor:** La salida `caudalActual` se conecta en el modelo acoplado a la entrada del Sensor de Flujo, que reportará el cambio medio segundo después de que el controlador emitió la orden.
4. **Tipos de datos:** Se utiliza $\mathbb{R}$ para los caudales (0 a 200 ml/h) y `{signal}` para el evento discreto de detención.

4. Módulo de Alarmas

El modelo **ModuloAlarmas** gestiona las notificaciones hacia el personal médico, emitiendo alertas y manteniendo un ciclo de retransmisión para las alarmas críticas no confirmadas.

atomic ModuloAlarmas is <X, Y, S, δ_{int} , δ_{ext} , λ , ta> where

S is

```

    tipoAlarma : {BAJA, MEDIA, CRITICA, NINGUNA};
    fase       : {OCIOSO, NOTIFICANDO, ESPERANDO_CONF, REPETIENDO};
     $\sigma$       : Time; -- control del tiempo de avance

```

end S

X is

```

    alarma           : {baja, media, critica}; -- niveles de alarma
    confirmacionEnfermero : {signal};          -- confirmacion manual

```

end X

Y is

```

    notificacionAlarma : Text; -- mensaje hacia el entorno exterior

```

end Y

$\delta_{ext}((tipoAlarma, fase, \sigma), e, (puerto, valor))$ is

```

     $\sigma = \sigma - e$ ;
    defcases
        tipoAlarma = valor; fase = NOTIFICANDO;  $\sigma = 0.0$ ;
        if (puerto = alarma)

```

```

            tipoAlarma = NINGUNA; fase = OCIOSO;  $\sigma = \text{infinity}$ ;
            if (puerto = confirmacionEnfermero)

```

```

        end defcases

```

end δ_{ext}

$\delta_{int}(tipoAlarma, fase, \sigma)$ is

```

    defcases
        fase = ESPERANDO_CONF;  $\sigma = 30.0$ ;
        if (tipoAlarma = CRITICA  $\wedge$  fase = NOTIFICANDO)

            fase = REPETIENDO;  $\sigma = 10.0$ ;
            if (tipoAlarma = CRITICA  $\wedge$  (fase = ESPERANDO_CONF  $\vee$  fase = REPETIENDO))

```

```

                tipoAlarma = NINGUNA; fase = OCIOSO;  $\sigma = \text{infinity}$ ;
                if (tipoAlarma = MEDIA  $\vee$  tipoAlarma = BAJA)

```

```

            end defcases

```

end δ_{int}

```

λ(tipoAlarma, fase, σ) is
    notificacionAlarma = "ALERTA: " + tipoAlarma;
end λ
ta(tipoAlarma, fase, σ) is
    σ;
end ta
end atomic

```

Figura 4: Especificación CML-DEVS: Módulo de Alarmas (Corregido)

Análisis del modelo

1. **Puertos de entrada:** El modelo recibe eventos por dos puertos: **alarma**, con valores enumerados {baja, media, critica} que unifican los tres niveles de alerta, y **confirmacionEnfermero**, de tipo **signal**, para el reconocimiento manual del personal médico.
2. **Estados transitorios** ($ta = 0$): Al recibir cualquier valor en el puerto **alarma**, la δ_{ext} actualiza el estado a **fase** = NOTIFICANDO y asigna $\sigma = 0$, forzando la ejecución inmediata de λ para que el mensaje salga al exterior, seguida de δ_{int} .
3. **Lógica de repetición para alarma crítica:** Luego de la primera notificación, δ_{int} transiciona a ESPERANDO_CONF con $\sigma = 30,0$ segundos. Si en ese lapso no llega la señal **confirmacionEnfermero**, pasa al estado REPETIENDO con ciclos ininterrumpidos de $\sigma = 10,0$ segundos, cumpliendo estrictamente los requisitos temporales 6 y 7 del proyecto.
4. **Alarmas no críticas:** Los valores **media** y **baja** se notifican una única vez; luego δ_{int} resetea el estado a OCIOSO con $\sigma = \infty$.
5. **Gestión de confirmación:** Si ingresa el evento **confirmacionEnfermero** mientras **tipoAlarma** = CRITICA, la δ_{ext} limpia el estado y detiene el ciclo de repetición asignando $\sigma = \infty$. Si no hay alarma crítica activa, la confirmación es simplemente ignorada manteniendo el cronograma de avance de tiempo intacto.

5. Controlador de Bomba

El modelo **ControladorBomba** es el componente central del sistema. Recibe órdenes médicas, monitorea el caudal real, detecta desvíos y condiciones de riesgo, y coordina las acciones sobre el actuador y el módulo de alarmas.

```

atomic ControladorBomba is <X, Y, S,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta> where
S is
    caudalObjetivo : ℝ; -- dosis indicada por ordenMedica (0-200 ml/h)
    caudalReal      : ℝ; -- ultima medicion del sensor de flujo
    fase : {OCIOSO, INFUNDIENDO, ALERTA_BOLSA, ALARMA_MEDIA, ALARMA_CRITICA};
    contadorMedia  : N; -- cuenta desvios consecutivos para alarma critica
    σ : Time; -- tiempo para la proxima transicion interna
    t_desvio       : Time; -- temporizador de desvio >10% (limite 5s)
    t_bolsa        : Time; -- temporizador tras fin de bolsa (limite 60s)

```

```

end S
X is
  ordenMedica      :  $\mathbb{R}$ ;      -- caudal objetivo
  sensorFlujo      :  $\mathbb{R}$ ;      -- caudal real medido
  finBolsa         : {signal};    -- bolsa casi vacia
  confirmacionEnfermero : {signal}; -- acuse de recibo de alarmas
end X
Y is
  ajustarCaudal    :  $\mathbb{R}$ ;      -- comando al actuador
  detenerBomba     : {signal};    -- parada de emergencia al actuador
  alarma           : {baja, media, critica}; -- alertas al ModuloAlarmas
  registrarEvento  : Text;      -- registro de auditoria
end Y
 $\delta_{ext}((\text{caudalObjetivo}, \text{caudalReal}, \text{fase}, \text{contadorMedia},$ 
   $\sigma, \text{t\_desvio}, \text{t\_bolsa}), e, (\text{puerto}, \text{valor}))$  is

  t_desvio = t_desvio - e;
  t_bolsa = t_bolsa - e;

  defcases
    caudalObjetivo = valor; fase = INFUNDIENDO;  $\sigma = 2.0$ ;
    if (puerto = ordenMedica  $\wedge$  valor > 0.0)

    caudalObjetivo = valor; fase = OCIOSO;  $\sigma = 0.0$ ;
    if (puerto = ordenMedica  $\wedge$  valor = 0.0)

    caudalReal = valor; t_desvio = 5.0;  $\sigma = \min(\sigma - e, 5.0)$ ;
    if (puerto = sensorFlujo  $\wedge$  t_desvio = infinity  $\wedge$ 
        abs(valor - caudalObjetivo) > 0.1 * caudalObjetivo)

    caudalReal = valor;  $\sigma = \min(\sigma - e, \text{t\_desvio})$ ;
    if (puerto = sensorFlujo  $\wedge$  t_desvio  $\neq$  infinity  $\wedge$ 
        abs(valor - caudalObjetivo) > 0.1 * caudalObjetivo)

    caudalReal = valor; t_desvio = infinity;
    contadorMedia = 0;  $\sigma = \sigma - e$ ;
    if (puerto = sensorFlujo  $\wedge$ 
        abs(valor - caudalObjetivo)  $\leq$  0.1 * caudalObjetivo)

    fase = ALERTA_BOLSA; t_bolsa = 60.0;  $\sigma = 0.0$ ;
    if (puerto = finBolsa)

    fase = OCIOSO;  $\sigma = 0.0$ ; contadorMedia = 0;
    if (puerto = confirmacionEnfermero  $\wedge$  fase = ALARMA_CRITICA)
  end defcases
end  $\delta_{ext}$ 
 $\delta_{int}(\text{caudalObjetivo}, \text{caudalReal}, \text{fase}, \text{contadorMedia}, \sigma, \text{t\_desvio}, \text{t\_bolsa})$  is
  defcases
    contadorMedia = contadorMedia + 1; fase = ALARMA_MEDIA; t_desvio = infinity;
     $\sigma = 0.0$ ;
    if (t_desvio  $\leq$  0.0  $\wedge$  contadorMedia  $\leq$  1)

```

```

    contadorMedia = 3; fase = ALARMA_CRITICA; t_desvio = infinity;  $\sigma$  = 0.0;
    if (t_desvio  $\leq$  0.0  $\wedge$  contadorMedia = 2)

caudalObjetivo = 0.0; fase = OCIOSO; t_bolsa = infinity;  $\sigma$  = 0.0;
    if (t_bolsa  $\leq$  0.0)

fase = INFUNDIENDO;  $\sigma$  = min(t_desvio, t_bolsa);
    if (fase = ALARMA_MEDIA)

fase = INFUNDIENDO;  $\sigma$  = min(t_desvio, t_bolsa);
    if (fase = ALERTA_BOLSA)

 $\sigma$  = min(t_desvio, t_bolsa);
    if (fase = INFUNDIENDO  $\wedge$  t_bolsa > 0.0  $\wedge$  t_desvio > 0.0)

 $\sigma$  = infinity;
    if (fase = ALARMA_CRITICA  $\wedge$  t_bolsa > 0.0)

 $\sigma$  = infinity;
    if (fase = OCIOSO  $\wedge$  t_bolsa > 0.0)
end defcases
end  $\delta_{int}$ 
 $\lambda$ (caudalObjetivo, caudalReal, fase, contadorMedia,  $\sigma$ , t_desvio, t_bolsa) is
    defcases
        ajustarCaudal = caudalObjetivo;
        if (fase = INFUNDIENDO)
        alarma = baja;
        if (fase = ALERTA_BOLSA)
        alarma = media;
        if (fase = ALARMA_MEDIA)
        alarma = critica; detenerBomba = signal;
        if (fase = ALARMA_CRITICA)
        detenerBomba = signal;
        if (fase = OCIOSO  $\wedge$  caudalObjetivo = 0.0)
    end defcases
    registrarEvento = "Evento de control ejecutado";
end  $\lambda$ 
ta(caudalObjetivo, caudalReal, fase, contadorMedia,  $\sigma$ , t_desvio, t_bolsa) is
     $\sigma$ ;
end ta
end atomic

```

Figura 5: Especificación CML-DEVS: Controlador de Bomba (Corregido)

Análisis del modelo

1. **Gestión de temporizadores concurrentes:** El modelo maneja múltiples límites de tiempo simultáneos (inicio en 3s, desvío en 5s, fin de bolsa en 60s). Para no perder consistencia temporal, en la δ_{ext} se resta el tiempo transcurrido e a todos los temporizadores en curso, y la función de avance de tiempo ta

siempre retorna la variable de control general σ , la cual se calcula utilizando la función *min()* entre los tiempos activos.

2. **Monitoreo de desvío sostenido:** Si la diferencia entre el caudal real y el objetivo supera el 10%, se inicia un temporizador `t_desvio = 5.0`. Es crítico destacar que este reloj solo se inicializa si previamente estaba apagado (`infinity`). Si el sensor sigue reportando fallas en los segundos posteriores, el reloj continúa su cuenta regresiva sin reiniciarse, garantizando que la alarma suene a los 5 segundos exactos.
3. **Criterio de Escalado y Seguridad:** A diferencia de delegar la lógica en las alarmas, el controlador acumula los desvíos en `contadorMedia`. Al alcanzar la tercera falla concurrente, la función λ emite la alarma crítica y, por el puerto `detenerBomba`, detiene físicamente el motor, asegurando la propiedad de seguridad (safety) del sistema.
4. **Fin de infusión:** Al recibir la señal `finBolsa`, el controlador transiciona a `ALERTA_BOLSA` y fuerza una salida inmediata para notificar la alarma baja. Inmediatamente después, retoma el estado de infusión dejando correr en paralelo el temporizador `t_bolsa = 60.0`. Cuando este tiempo real se agota, el sistema se detiene por completo.

6. Generador de Confirmación de Enfermero

El modelo **GeneradorConfirmacion** es un modelo atómico autónomo que genera periódicamente eventos `confirmacionEnfermero`, simulando el reconocimiento manual de alarmas por parte del personal médico.

```
atomic GeneradorConfirmacion is <Y, S,  $\delta_{int}$ ,  $\lambda$ , ta> where
S is
   $\sigma$  : Time; -- tiempo hasta la siguiente confirmacion
end S
Y is
  confirmacionEnfermero : {signal}; -- evento de confirmacion
end Y
 $\delta_{int}(\sigma)$  is
  -- A definir en la implementacion: intervalo entre confirmaciones
   $\sigma$  = next_confirmation_time();
end  $\delta_{int}$ 
 $\lambda(\sigma)$  is
  confirmacionEnfermero = signal;
end  $\lambda$ 
ta( $\sigma$ ) is
   $\sigma$ ;
end ta
end atomic
```

Figura 6: Especificación CML-DEVS: Generador de Confirmación de Enfermero

7. Gerador de Fin de Bolsa

El modelo **GeneradorFinBolsa** es un modelo atómico autónomo que genera el evento **finBolsa** cuando la bolsa de medicación está próxima a agotarse. El momento del evento se determina en la implementación en base al volumen inicial.

```

atomic GeneradorFinBolsa is <Y, S,  $\delta_{int}$ ,  $\lambda$ , ta> where
S is
     $\sigma$  : Time;  -- tiempo hasta el proximo fin de bolsa
end S
Y is
    finBolsa : {signal};
end Y
 $\delta_{int}(\sigma)$  is
    -- A definir en la implementacion: basado en volumen inicial
     $\sigma$  = next_finBolsa_logic();
end  $\delta_{int}$ 
 $\lambda(\sigma)$  is
    finBolsa = signal;
end  $\lambda$ 
ta( $\sigma$ ) is
     $\sigma$ ;
end ta
end atomic

```

Figura 7: Especificación CML-DEVS: Generador de Fin de Bolsa

8. Modelo Acoplado

La Figura 8 muestra el modelo acoplado del sistema, especificando los puertos y las conexiones entre los modelos atómicos.

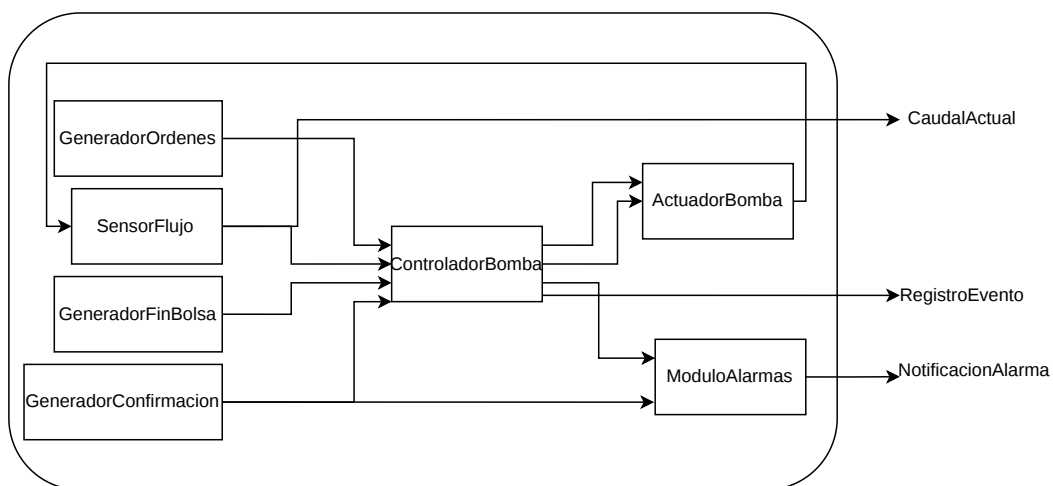


Figura 8: Modelo Acoplado del Sistema de Bomba de Infusión